External App Integration Guide

> **Audience**: External developers integrating their applications with Atlan

- Overview
  - Mode 1: Embedded (Production)
  - Mode 2: Standalone (Local Development)
- Quick Start with Atlan Auth SDK
  - Installation
  - SDK Features
- Complete Example
- How It Works
  - Embedded Mode (Inside Atlan)
  - Standalone Mode (Local Development)
- Handling Loading States
- API Reference
  - Quick Reference
- Framework Examples
  - React
  - Vue 3
- Local Development Setup
- Troubleshooting
  - Issue: Redirect loop in standalone mode
  - Issue: "Not authenticated" errors
  - Issue: TypeScript errors with keycloak-js
- Security
  - What's Secure?
  - What to Avoid?
  - Content Security Policy (CSP) Headers
- Next Steps
- Comparison: SDK vs Manual Integration

Overview

Atlan can embed external applications as tabs in the left navigation sidebar. Your application works in two modes:

Mode 1: Embedded (Production)

- App runs inside Atlan as an iframe
- Receives JWT token via `postMessage` API
- No redirect or login flow needed
- User context automatically provided

Mode 2: Standalone (Local Development)

- App runs independently (e.g., `localhost:8000`)
- Uses Keycloak for authentication (redirects to Atlan login)
- Fast local iteration without running Atlan frontend
- No OAuth credentials needed

Quick Start with Atlan Auth SDK

The easiest way to integrate is using the official SDK. It handles authentication automatically.

Installation

Option A: CDN (No build required)

```
<script src="https://unpkg.com/@atlanhq/atlan-auth@latest/dist/atlan-auth.umd.min.js"></script>
<script>
    const { AtlanAuth } = window.AtlanAuthSDK

    const atlan = new AtlanAuth({
        origin: 'https://your-tenant.atlan.com',
    })

    atlan.init().then(() => {
        console.log('✅ Authenticated!', atlan.getUser())
    })
</script>

```

Option B: npm Install

```
npm install @atlanhq/atlan-auth keycloak-js

```

```
import { AtlanAuth } from '@atlanhq/atlan-auth'

const atlan = new AtlanAuth({
    origin: 'https://your-tenant.atlan.com',
})

await atlan.init()
console.log('User:', atlan.getUser())

// Make API calls
const assets = await atlan.api.get('/api/meta/search/indexsearch')

```

SDK Features

- **Zero Config**: Just provide your Atlan origin URL
- **Automatic Authentication**: Detects embedded vs standalone mode
- **Keycloak Integration**: Secure PKCE flow (no secrets in browser)
- **Auto Token Refresh**: No manual token management
- **Type-safe API client**: Built-in fetch wrapper with auth
- **Small bundle**: ~52KB gzipped (includes Keycloak)

Complete Example

```
<!DOCTYPE html>
<html>
    <head>
        <title>My Atlan App</title>
    </head>
    <body>
        <div id="app">Loading...</div>

        <script src="https://unpkg.com/@atlanhq/atlan-auth@latest/dist/atlan-auth.umd.min.js"></script>
        <script>
            const { AtlanAuth } = window.AtlanAuthSDK

            const atlan = new AtlanAuth({
                origin: 'https://your-tenant.atlan.com',
                onReady: (context) => {
                    console.log('✅ SDK Ready!')
                    document.getElementById('app').innerHTML = `
                        <h1>Welcome, ${context.user.username}!</h1>
                        <p>Mode: ${context.mode}</p>
                    `
                },
                onError: (error) => {
```

```
                    console.error('❌ Error:', error)
                    document.getElementById('app').innerHTML = `
                        <h1>Authentication failed</h1>
                        <p>${error.message}</p>
                    `
                },
                debug: true,
            })

            atlan.init()
        </script>
    </body>
</html>

```

How It Works

Embedded Mode (Inside Atlan)

```
┌──────────────────────────────────────┐
│ Atlan Frontend                       │
│                                      │
│  ┌──────────────────────────────┐    │
│  │ Your App (iframe)            │    │
│  │                              │    │
│  │  1. SDK detects iframe       │    │
│  │  2. Sends IFRAME_READY ──────┼────┼──┐
│  │                              │    │  │
│  │  3. ◀── Receives token       │◀───┼──┘
│  │     (via postMessage)        │    │
│  │                              │    │
│  │  4. ✅ Instant authentication │    │
│  └──────────────────────────────┘    │
└──────────────────────────────────────┘

```

Standalone Mode (Local Development)

```
┌──────────────────────────────────────┐
│ Your App (localhost:8000)            │
│                                      │
│  1. SDK detects standalone mode      │
│  2. Silent SSO check (instant if logged in!)
│  3. Redirects to Keycloak if needed ─┼──┐
└──────────────────────────────────────┘  │
                                          │
                                          │
┌──────────────────────────────────────┐  │
│ Atlan Keycloak Login                 ◀─┘
│                                      │
│  4. You log in with credentials      │
│  5. Redirects back with auth code ───┼──┐
└──────────────────────────────────────┘  │
                                          │
                                          │
┌──────────────────────────────────────┐  │
│ Your App (localhost:8000)            ◀─┘
│                                      │
│  6. SDK exchanges code for token     │
│  7. ✅ Authenticated                  │
└──────────────────────────────────────┘

```

Key Points:

- **Silent SSO**: Returning users authenticate instantly—no redirect!
- **No secrets needed**: Uses Keycloak's PKCE flow (secure for browsers)
- **Automatic redirect**: SDK handles the entire flow
- **Same user session**: You log in with your actual Atlan account

---

Handling Loading States

Prevent a flash of content before authentication completes using CSS-first loading patterns:

```html
<head>
    <style>
        /* Hide app until authenticated */
        body.loading #app {
            display: none;
        }
        body.loading::before {
            content: '🔐 Authenticating...';
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
        }
    </style>
</head>
<body class="loading">
    <div id="app">
        <!-- Your app content -->
    </div>

    <script>
        const { AtlanAuth } = window.AtlanAuthSDK

        const atlan = new AtlanAuth({
            origin: 'https://your-tenant.atlan.com',
            onReady: () => {
                document.body.classList.remove('loading')
            },
        })

        atlan.init()
    </script>
</body>
```

Why CSS-first? CSS loads before JavaScript, so the loading state is visible from the first paint—no flash!

API Reference

Quick Reference

```
// Initialize
await atlan.init()

// Get data
const token = atlan.getToken()
const user = atlan.getUser()
const tenant = atlan.getTenant()
const mode = atlan.getMode() // 'embedded' or 'standalone'

// Make API calls
const response = await atlan.api.get('/api/service/whoami')
const data = await atlan.api.post('/api/meta/search', { query: '...'
})
```

Framework Examples

React

```
import { AtlanAuth } from '@atlanhq/atlan-auth'
import { useEffect, useState } from 'react'

function App() {
    const [atlan, setAtlan] = useState(null)
    const [user, setUser] = useState(null)

    useEffect(() => {
        const sdk = new AtlanAuth({
            origin: 'https://your-tenant.atlan.com',
            onReady: (context) => {
                setUser(context.user)
```

```
            },
        })

        sdk.init().then(() => setAtlan(sdk))
    }, [])

    if (!user) return <div>Loading...</div>

    return <h1>Welcome, {user.username}!</h1>
}

```

Vue 3

```
<template>
    <div v-if="user">
        <h1>Welcome, {{ user.username }}!</h1>
    </div>
    <div v-else>Loading...</div>
</template>

<script setup>
import { AtlanAuth } from '@atlanhq/atlan-auth'
import { ref, onMounted } from 'vue'

const user = ref(null)

onMounted(async () => {
    const atlan = new AtlanAuth({
        origin: 'https://your-tenant.atlan.com',
        onReady: (context) => {
            user.value = context.user
        },
    })

    await atlan.init()
})
</script>

```

Local Development Setup

1. **Start your app**:

```
# Any local server works
python3 -m http.server 8000
# or
npx serve .
# or
npm run dev

```

2. **Open in browser**: `http://localhost:8000`
3. **First time**: SDK will redirect you to Atlan login page
4. **Log in**: Use your Atlan credentials
5. **Redirected back**: SDK automatically completes authentication
6. **Start developing**: Your app is now authenticated!

Troubleshooting

Issue: Redirect loop in standalone mode

⚠️ **Cause**: Atlan instance not accessible at the configured origin URL.

Solution:

1. Ensure Atlan is running at the origin URL
2. Check that you can access `{origin}/auth` in your browser

3. Verify no CORS issues

Issue: "Not authenticated" errors

⚠️ **Cause**: SDK not initialized before making API calls.

Solution:

```
// ❌ Wrong
const atlan = new AtlanAuth({ origin: '...' })
atlan.api.get('/api/...') // SDK not ready!

// ✅ Correct
const atlan = new AtlanAuth({ origin: '...' })
await atlan.init() // Wait for initialization
atlan.api.get('/api/...') // Now safe

```

Issue: TypeScript errors with keycloak-js

⚠️ **Cause**: Missing peer dependency.

Solution:

```
npm install keycloak-js

```

Security

What's Secure?

- ✅ **No secrets in browser**: Uses Keycloak PKCE (no client secret needed)
- ✅ **Secure token storage**: Managed by Keycloak, not localStorage
- ✅ **Automatic token refresh**: Tokens refresh before expiration
- ✅ **Origin validation**: postMessage validates message origins

What to Avoid?

- ❌ Don't hardcode tokens in your code
- ❌ Don't store tokens in localStorage manually
- ❌ Don't skip the `await atlan.init()` step
- ❌ Don't commit .env files with credentials

Content Security Policy (CSP) Headers

When deploying external iframe integrations, ensure your Content-Security-Policy headers allow iframe embedding:

Required CSP Directives:

```
Content-Security-Policy:
  frame-src 'self' https://*.atlan.com https://your-external-app.com;
  frame-ancestors 'self' https://*.atlan.com;

```

For Development:

```
Content-Security-Policy:
  frame-src 'self' http://localhost:* https://*.atlan.com;
  frame-ancestors 'self' http://localhost:*;

```

Important:

- Add all external app domains to `frame-src`
- External apps must set `frame-ancestors` to allow Atlan domains
- Use specific domains instead of `*` in production
- Test CSP headers in staging before production deployment

Next Steps

1. ✅ Install the SDK
2. ✅ Add initialization code
3. ✅ Test locally (standalone mode)
4. ✅ Test in Atlan (embedded mode)
5. ✅ Deploy your app

Need help? Open a ticket or contact Atlan support

Comparison: SDK vs Manual Integration

Feature	SDK	Manual
Lines of Code	~10 lines	~200+ lines
Auth Setup	Zero config	OAuth client + postMessage
Token Refresh	Automatic	Manual implementation
Mode Detection	Automatic	Manual iframe detection
Error Handling	Built-in	Manual implementation
TypeScript	Full support	Manual types
Bundle Size	52KB (includes Keycloak)	Varies
Maintenance	SDK handles updates	You maintain

Recommendation: Always use the SDK unless you have specific requirements that prevent it.